

Elman, Jordan and Multi-Recurrent Networks for One-Step-Ahead Time Series Forecasting

Leesan Moodley

Student number: 26186152

Department of Mathematical Sciences

Stellenbosch University, Stellenbosch, South Africa

26186152@sun.ac.za

Code: <https://github.com/Leesan0205/MLL3.git>

Abstract—Simple recurrent neural networks differ only in the information fed back from one time step to the next. This report compares the Elman network, the Jordan network and the multi-recurrent network (MRN) for one-step-ahead forecasting. The comparison uses five univariate time series. The series are Airline Passengers, Monthly Sunspots, Melbourne daily minimum temperatures, the chaotic Mackey-Glass series and the Standard and Poor's 500 (S&P 500) index. The augmented Dickey-Fuller and Kwiatkowski-Phillips-Schmidt-Shin tests identified two non-stationary series. Both non-stationary series were log-transformed and differenced. Random search over 24 configurations selected the hyper-parameters of each network. Five-fold expanding-window cross-validation with two seeds scored each configuration. Each selected network was evaluated on the final 20% of every series over 10 random seeds. The MRN achieved the lowest root mean squared error on the Airline, Temperature and Mackey-Glass series. The Friedman test found significant differences between the networks on all three of these series. The MRN also obtained the best overall average rank of 1.70. The Elman network ranked second with 2.12. The Jordan network ranked third with 2.78. No significant differences were found on the Sunspots and S&P 500 series. The MRN is recommended, although the margin of the MRN over the Elman network was small.

Index Terms—recurrent neural networks, Elman network, Jordan network, multi-recurrent network, time series forecasting, cross-validation

I. INTRODUCTION

Simple recurrent neural networks (SRNs) extend the feed-forward multilayer perceptron with feedback connections. The feedback connections store a summary of past inputs in context units. Three classic SRN variants differ only in the information fed back. The Elman network feeds back the hidden layer. The Jordan network feeds back the output layer. The multi-recurrent network (MRN) feeds back both layers into several memory banks. Each memory bank has an individual rate of decay. The three variants share the same input and output layers. A comparison of the three variants therefore isolates the effect of the feedback structure on forecasting accuracy.

The goal of this assignment was to implement the three SRNs and to apply the SRNs to five time series. The five series differ in length, sampling frequency and stationarity. The assignment determined the most accurate SRN for one-step-ahead forecasting on each dataset and overall. The report is governed by the principle that the evaluation respects the

order of time. No model sees an observation that lies after the forecast point during training or selection.

The approach started by describing each dataset and by testing each dataset for stationarity. The stationarity results determined the pre-processing of each series. The three SRNs were implemented from the defining equations. All three SRNs shared the same output layer, loss function and optimiser. The feedback structure was therefore the only difference between the SRNs. Random search with expanding-window cross-validation (CV) selected the hyper-parameters. The selected models were evaluated on an untouched test set over 10 random seeds. Each SRN was also compared against a naive persistence baseline. The persistence baseline forecasts the next value as the most recent observation. The Friedman, Nemenyi and Wilcoxon tests assessed the statistical significance of the differences between the SRNs. The principal finding was that the MRN performed best. The Elman network followed closely. The Jordan network was consistently the weakest.

The remainder of the report is organised as follows. Section II reviews the three SRNs and CV for time series. Section III describes the datasets. Section IV presents the pre-processing. Section V describes the implementation. Section VI defines the empirical procedure. Section VII presents the results. Section VIII discusses the results. Section IX concludes the report.

II. BACKGROUND

This section provides the theoretical background of the study. The section first describes the three SRNs. The section then explains why time series data requires a specialised form of CV.

A. Simple Recurrent Neural Networks

All three SRNs process an input sequence one time step at a time. The input at time step t is the scalar x_t . The hidden state at time step t is the vector $\mathbf{h}_t$ with n_h hidden units. The output at time step t is the scalar y_t . The hidden layer of each SRN uses the hyperbolic tangent activation function. The output layer of each SRN is linear and computes

$$y_t = \mathbf{V}\mathbf{h}_t + \beta, \quad (1)$$

where $\mathbf{V}$ is the hidden-to-output weight matrix and β is the output bias.

1) *Elman Network*: Elman proposed a network that copies the hidden state into context units. The context vector at time step t is $\mathbf{c}_t = \mathbf{h}_{t-1}$. The context units act as additional inputs to the hidden layer. The Elman network computes the hidden state as

$$\mathbf{h}_t = \tanh(\mathbf{W}x_t + \mathbf{U}\mathbf{c}_t + \mathbf{b}), \quad (2)$$

where $\mathbf{W}$ is the input-to-hidden weight matrix, $\mathbf{U}$ is the context-to-hidden weight matrix and $\mathbf{b}$ is the hidden bias vector. The context vector carries a learned summary of the past with n_h dimensions.

2) *Jordan Network*: Jordan proposed a network that feeds back the output instead of the hidden state. The state unit $s_t = y_{t-1}$ enters the hidden layer. The Jordan network computes the hidden state as

$$\mathbf{h}_t = \tanh(\mathbf{W}x_t + \mathbf{G}s_t + \mathbf{b}), \quad (3)$$

where $\mathbf{G}$ is the state-to-hidden weight matrix. The memory of the Jordan network therefore consists of a single number. The single number is the previous forecast of the network. The original formulation of Jordan also included a self-connection on the state unit. The present study used the plain copy to keep the three SRNs distinct. The MRN covers decaying self-connections instead.

3) *Multi-Recurrent Network*: Ulbricht proposed the MRN. Binner *et al.* later applied the MRN to economic forecasting. The MRN combines the Elman and Jordan feedback paths. The MRN also adds n_b memory banks with self-recurrent links. Bank j has a fixed decay rate $a_j \in [0, 1)$. The hidden memory $\mathbf{m}_{j,t}$ and the output memory $r_{j,t}$ of bank j are updated as

$$\mathbf{m}_{j,t} = a_j \mathbf{m}_{j,t-1} + (1 - a_j) \mathbf{h}_{t-1}, \quad (4)$$

$$r_{j,t} = a_j r_{j,t-1} + (1 - a_j) y_{t-1}. \quad (5)$$

The vector $\mathbf{z}_t$ concatenates the hidden and output memories of all banks. The MRN computes the hidden state as

$$\mathbf{h}_t = \tanh(\mathbf{W}x_t + \mathbf{Q}\mathbf{z}_t + \mathbf{b}), \quad (6)$$

where $\mathbf{Q}$ is the memory-to-hidden weight matrix. Each bank computes an exponential moving average of past activations. A bank with a small decay rate reacts quickly to new activations. A bank with a large decay rate keeps a longer and smoother memory. A single bank with a decay rate of zero stores exactly the pair $(\mathbf{h}_{t-1}, y_{t-1})$. The single-bank MRN is therefore the union of the Elman and Jordan feedback. For an equal number of hidden units, the MRN has $n_b(n_h + 1)n_h$ recurrent weights. The Elman network has n_h^2 recurrent weights. The Jordan network has n_h recurrent weights.

All three SRNs are trained with backpropagation through time over the input window. Gradients propagated through many recurrent steps tend to vanish or to explode. Gradient-norm clipping limits exploding gradients.

B. Cross-Validation for Time Series

Standard k -fold CV assigns observations to k folds at random. Random assignment causes two problems for time series [7], [8]. First, a model trains on observations that lie after the validation observations. The model then uses information from the future. Future information is never available at forecast time. Second, consecutive observations are autocorrelated. Neighbouring samples share most of the inputs after sliding-window framing. Random folds then place near-duplicates of every validation sample in the training set. The resulting error estimate is optimistic. Bergmeir and Benítez compared CV schemes for time series predictors empirically.

The recommended alternative is rolling-origin evaluation [7], [8]. Rolling-origin evaluation is also called forward chaining or expanding-window CV. The series is cut at several forecast origins. For each origin, the model trains on all observations before the origin. The model then validates on the block that follows the origin. The training sets grow from fold to fold. The temporal order is never violated. Cerqueira *et al.* found that out-of-sample schemes gave the most reliable estimates for non-stationary real-world series.

III. DATASETS

This section describes the five time series used in the study. The section first describes the origin and properties of each series. The section then assesses the stationarity of each series.

A. Dataset Descriptions

The five series cover trending, seasonal, cyclical, chaotic and financial behaviour. Table I summarises the properties of the five series.

Fig. 1 plots each series. The shaded region in Fig. 1 marks the final 20% of each series. The final 20% forms the held-out test set. The five series are described below.

- *Airline Passengers*: The series contains monthly totals of international airline passengers in thousands from 1949 to 1960. The series has an upward trend. The amplitude of the seasonal pattern grows with the level of the series.
- *Monthly Sunspots*: The series contains the mean monthly sunspot number from 1749 to 1983. The series is cyclical with a period of approximately 132 months. The height of each cycle is irregular.
- *Daily Minimum Temperatures*: The series contains the daily minimum temperature in Melbourne, Australia from 1981 to 1990. The series has a strong annual cycle and high day-to-day noise.
- *Mackey-Glass*: Mackey and Glass proposed a delay differential equation that produces a chaotic series. The equation is

$$\frac{dv}{dt} = \frac{0.2v(t - \tau)}{1 + v(t - \tau)^{10}} - 0.1v(t),$$

where $v(t)$ is the state at time t and $\tau = 17$ is the delay. The equation was integrated with the Euler method using a step size of $\delta = 0.01$ and a constant history of $v = 1.2$. The integrated series was sampled once per time unit. The

TABLE I
SUMMARY OF THE FIVE DATASETS

Dataset	Frequency	Span	Length	Missing	Mean	Std. dev.	Min	Max
Airline	Monthly	1949-01–1960-12	144	0	280.30	119.97	104.0	622.0
Sunspots	Monthly	1749-01–1983-12	2820	0	51.27	43.45	0.0	253.8
Temperature	Daily	1981-01–1990-12	3652	2	11.18	4.07	0.0	26.3
Mackey-Glass	Unit time step	$t = 0-1499$	1500	0	0.930	0.227	0.419	1.317
S&P 500	Monthly	1950-01–2024-12	900	0	845.8	1169.0	16.9	6010.9

TABLE II
STATIONARITY TESTS ON THE DEVELOPMENT PORTION OF EACH SERIES

Dataset	Series	ADF	p	KPSS	p	Verdict
Airline	raw	-0.36	0.917	1.680	≤ 0.01	Non-stationary
Airline	log-diff	-2.64	0.086	0.018	≥ 0.10	Inconclusive
Airline	+ seasonal diff (not used)	-3.95	0.002	0.084	≥ 0.10	Stationary
Sunspots	raw	-7.94	< 0.001	0.176	≥ 0.10	Stationary
Temperature	raw	-4.10	< 0.001	0.069	≥ 0.10	Stationary
Mackey-Glass	raw	-10.97	< 0.001	0.014	≥ 0.10	Stationary
S&P 500	raw	-0.36	0.917	3.086	≤ 0.01	Non-stationary
S&P 500	log-diff	-10.24	< 0.001	0.104	≥ 0.10	Stationary

first 500 samples were discarded as transient. The noise-free series shows how well each SRN learns deterministic nonlinear dynamics.

- *Standard and Poor's 500*: The series contains the monthly average of the S&P 500 stock index from 1950 to 2024. The logarithmic axis of Fig. 1 shows an approximately straight line for the S&P 500. The S&P 500 series therefore grows approximately exponentially.

The Airline, Sunspots and Temperature series were obtained from the Brownlee repository. All raw files are stored in the code repository. The stored files keep the results independent of future revisions of the sources.

B. Stationarity

Two complementary tests assessed stationarity at the 5% significance level. Both tests used only the development portion of each series. The development portion is the first 80% of each series. The test set therefore had no influence on any pre-processing decision. The augmented Dickey-Fuller (ADF) test has a unit root as the null hypothesis. A unit root implies non-stationarity. The Akaike information criterion selected the lag order of the ADF test. The Kwiatkowski-Phillips-Schmidt-Shin (KPSS) test has level stationarity as the null hypothesis. A series was labelled stationary when the ADF test rejected and the KPSS test did not reject. A series was labelled non-stationary when the KPSS test rejected and the ADF test did not reject. KPSS p -values are interpolated from a lookup table. The KPSS p -values are therefore only known within the range $[0.01, 0.10]$. Table II reports the test statistics and the p -values.

Table II confirms the visual impression of Fig. 1. The raw Airline and S&P 500 series are non-stationary. The ADF test could not reject a unit root for either series with a p -value of 0.917. The KPSS test rejected stationarity for both series with p -values of at most 0.01. The log-differenced S&P 500 returns are stationary. The log-differenced Airline series is borderline. The KPSS test did not reject stationarity for the

log-differenced Airline series. The ADF test rejected only at the 10% level with a p -value of 0.086. The remaining non-stationarity is the seasonal pattern visible in the bottom-right panel of Fig. 1.

An additional lag-12 seasonal difference made the Airline series stationary under both tests. The seasonal difference was not used for two reasons. First, the seasonal difference would remove another 12 of only 144 observations. Second, the SRNs receive up to two years of past monthly values as input. The SRNs therefore model the seasonality directly from the input. The Sunspots, Temperature and Mackey-Glass series are stationary under both tests. The strong cycles of the three series repeat around a constant mean.

IV. DATA PRE-PROCESSING

This section describes the pre-processing applied to each series. Every data-dependent pre-processing step was fitted on training data only. Every transform was inverted before errors were computed. The section covers missing values, transforms, scaling and supervised framing in the order of application.

A. Missing Values

The temperature file omits 31 December in the leap years 1984 and 1988. The series was re-indexed to a complete daily calendar. The re-indexing made the two gaps explicit. Linear interpolation between neighbouring days filled the two gaps. Skipping the gaps instead would shift the annual cycle by one day. Skipping the gaps would also break the equal spacing assumed by the SRNs. None of the other series had missing values.

B. Log and Differencing Transforms

The Airline and S&P 500 series were log-transformed. The variability of both series grows with the level of the series, as Fig. 1 shows. Both log series were then first-differenced to remove the trend. The resulting series are monthly growth rates. A network trained on raw levels has to extrapolate beyond the training range during testing. The hyperbolic tangent function is bounded. A network with bounded hidden units therefore struggles to produce values outside the training range. The stationary series were left untransformed.

Let o_t denote the observed value at time step t . Let u_t denote the transformed series and $\hat{u}_t$ the predicted change in log level. The transforms were inverted exactly for one-step-ahead forecasts. The forecast on the original scale is

$$\hat{o}_t = \exp(\log o_{t-1} + \hat{u}_t). \quad (7)$$

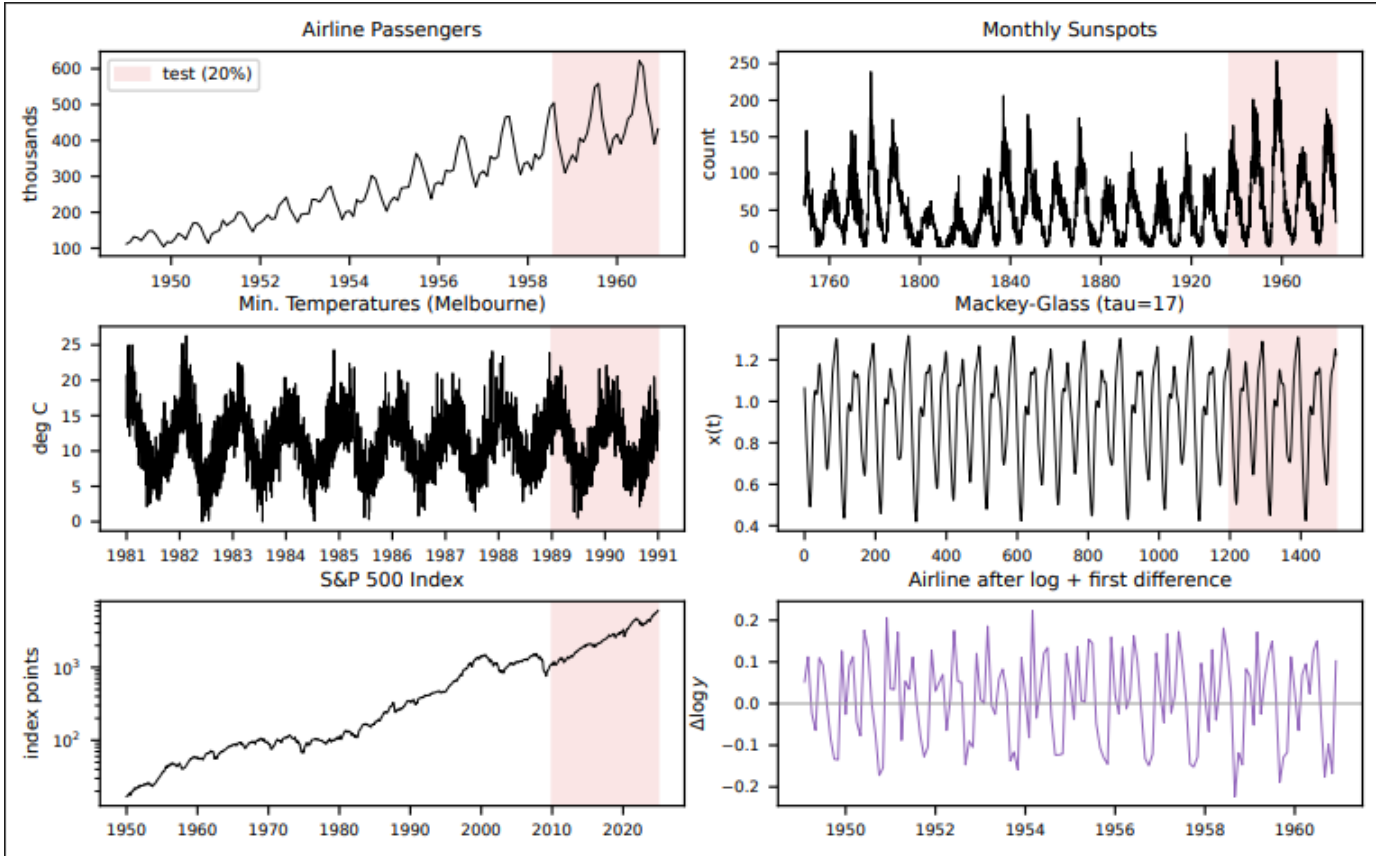


Fig. 1. The five time series and the transformed Airline series. Shading marks the held-out test set.

The inversion is legitimate because o_{t-1} is observed at the forecast origin.

C. Scaling

The transformed series was standardised to zero mean and unit variance. Standardisation used only the observations available up to the last training target of the current fit. No statistics from validation or test data were therefore used. Standardisation was preferred over min-max scaling. Later data easily exceeds a min-max range fitted on training data. Standardisation imposes no hard bounds.

D. Supervised Framing

A sliding window of length n_w converted the scaled series into input-target pairs. The window length n_w is a hyper-parameter. The input sequence for target t is $(u_{t-n_w}, \dots, u_{t-1})$ after scaling. The SRN made one prediction per input step. At every step within the window, the target was the next value of the series. The final step therefore gave the one-step-ahead forecast of u_t . The all-step supervision provided n_w training signals per window. The all-step supervision also gave the fed-back outputs of the Jordan network a meaningful target. The recurrent state was reset to zero at the start of every window.

Let d denote the differencing order. The first forecast target of every series was fixed at index $24 + d$. The value 24 is

the largest candidate window length. Every configuration was therefore scored on exactly the same targets.

V. IMPLEMENTATION

The SRNs were implemented in Python 3 with PyTorch. The built-in recurrent layers of PyTorch were not used. Each SRN cell is a custom module. Each module evaluates Equations (1) to (6) inside an explicit loop over time steps. Unit tests compared each cell against an independent NumPy implementation of the cell equations. The unit tests also checked that each cell fitted a sinusoid. The statsmodels library performed the stationarity tests. The SciPy library performed the statistical tests.

Let n_i denote the number of inputs to a layer. The weights were initialised with the PyTorch default. The default draws each weight from a uniform distribution over the range $[-1/\sqrt{n_i}, 1/\sqrt{n_i}]$. All random seeds were fixed. The single command `python run_all.py` reproduces every number, table and figure in the report.

VI. EMPIRICAL PROCEDURE

This section defines the empirical procedure of the study. The section describes the data splits, the training of the SRNs and the hyper-parameter search. The section then defines the performance measures and the statistical analysis.

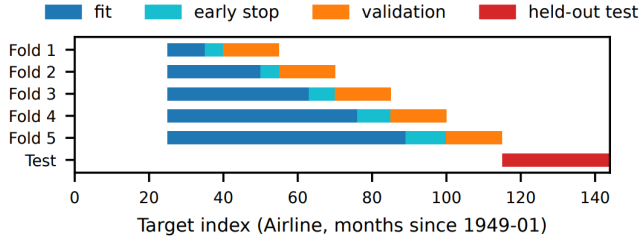


Fig. 2. Expanding-window cross-validation folds for the Airline series

A. Data Splits and Cross-Validation

Early stopping ends training once the loss on a held-out set stops improving. The data-splitting protocol consisted of three layers. Fig. 2 illustrates the three layers.

- 1) The final 20% of each series was held out as the test set. The test sets contain 29, 564, 730, 300 and 180 targets for the Airline, Sunspots, Temperature, Mackey-Glass and S&P 500 series. The test set was used only once after every choice had been fixed.
- 2) Five-fold expanding-window CV split the remaining development targets. Let n_d denote the number of development targets. The validation block contains $\lfloor n_d/6 \rfloor$ targets. Fold i validated on the i -th of the last five blocks. Fold i trained on every target before the validation block. The splits were never shuffled. No gap between training and validation was required. The inputs of a validation window are past observations. Past observations are available at the forecast origin. The targets of the training and validation sets never overlap.
- 3) The last 15% of the targets in each training set was reserved for early stopping. The reserved targets are the chronologically latest targets of the training set. The CV validation block therefore only scored a configuration. The CV validation block never influenced training.

B. Training and Regularisation

All three SRNs used the same output layer from Equation (1). All three SRNs minimised the mean squared error (MSE) on the scaled series. The Adam optimiser updated the weights with mini-batches of 64 windows. Shuffling windows between mini-batches does not violate the order of time. Each window is a self-contained sample with an input that precedes the target. The boundary between training and validation data is chronological. Four measures controlled the bias-variance trade-off:

- *Early stopping*: The MSE on the early-stopping set was computed after each epoch. Training stopped after 15 epochs without improvement. The weights of the best epoch were restored. The maximum number of epochs was 300 for the Airline, Sunspots, Temperature and S&P 500 series. The noise-free Mackey-Glass series was allowed up to 1000 epochs. A preliminary run with a limit

TABLE III
HYPER-PARAMETER SEARCH SPACE

Hyper-parameter	Values
Hidden units n_h	4, 8, 16, 32
Window length n_w	6, 12, 24
Learning rate	10^{-3} , 3×10^{-3} , 10^{-2}
Weight decay	0, 10^{-5} , 10^{-4} , 10^{-3}
Decay rates $\{a_j\}$	$\{0.25, 0.5, 0.75\}$, $\{0, 0.5, 0.9\}$, $\{0, 0.25, 0.5, 0.75\}$, $\{0.5\}$
Batch size	64
Maximum epochs	300; 1000 for Mackey-Glass
Patience	15
Clipping norm	1.0
Early-stopping fraction	0.15

of 300 epochs ended every Mackey-Glass run while the loss was still decreasing.

- *Weight decay*: The weight-decay term of Adam applied an L_2 penalty to the weights. CV chose the strength of the penalty.
- *Gradient-norm clipping*: The global gradient norm was clipped to 1.0. Clipping prevents exploding gradients in backpropagation through time.
- *Capacity selection*: CV chose the number of hidden units and the window length. Capacity selection guards against both underfitting and overfitting.

The comparison with the persistence baseline diagnosed underfitting. The gap between the training and validation loss curves diagnosed overfitting.

C. Hyper-Parameter Search

Table III lists the search space. Random search selected the hyper-parameters with a budget of 24 configurations per SRN and dataset. Bergstra and Bengio showed that random search covers the important dimensions more efficiently than grid search. The full grid contains 144 base configurations. The same 24 base configurations were drawn for all three SRNs of a dataset. The shared configurations ensured a fair comparison. The MRN additionally drew one set of decay rates per configuration. Each configuration was trained on the five CV folds with the two seeds 0 and 1. The two tuning seeds reduced the chance that a single lucky initialisation decided the selection. The mean validation root mean squared error (RMSE) over the 10 training runs scored each configuration. The configuration with the lowest mean validation RMSE was selected. The RMSE was computed on the original scale.

D. Performance Measures

Each selected configuration was retrained on the full development set with 10 seeds. The last 15% of the development set again served early stopping. The seeds varied the weight initialisation and the mini-batch order. Each retrained model forecast the test set one step ahead. Let T denote the set of test

time steps. Three measures assessed accuracy on the original scale. The RMSE is computed as

$$\sqrt{\frac{1}{|T|} \sum_{t \in T} (o_t - \hat{o}_t)^2}. \quad (8)$$

The mean absolute error (MAE) is computed as

$$\frac{1}{|T|} \sum_{t \in T} |o_t - \hat{o}_t|. \quad (9)$$

The symmetric mean absolute percentage error (sMAPE) is computed as

$$\frac{100}{|T|} \sum_{t \in T} \frac{2 |o_t - \hat{o}_t|}{|o_t| + |\hat{o}_t|}. \quad (10)$$

A term of the sMAPE is defined as zero when $o_t = \hat{o}_t = 0$ [?], [?]. The zero case occurs in the Sunspots series. The RMSE is the primary measure and penalises large errors. The MAE is robust to large errors. The sMAPE is scale-free and allows comparison across datasets. The naive persistence forecast $\hat{o}_t = o_{t-1}$ served as the baseline. A model that does not beat persistence has learnt nothing beyond the last observation.

E. Statistical Analysis

The statistical analysis followed the methodology of Demšar. For each dataset, each of the 10 seeds formed a block. The test RMSEs of the three SRNs were ranked within each block. The Friedman test checked whether the average ranks differ. The Nemenyi post-hoc test followed whenever the Friedman test rejected at the significance level $\alpha = 0.05$. The Nemenyi test declared a pair of methods different when the average ranks differed by more than the critical difference (CD). The CD is computed as

$$q_\alpha \sqrt{\frac{n_m(n_m + 1)}{6n_s}}, \quad (11)$$

where n_m is the number of methods and n_s is the number of blocks. The value q_α is the critical value of the studentised range statistic divided by $\sqrt{2}$.

Pairwise two-sided Wilcoxon signed-rank tests complemented the Nemenyi test. The Holm correction adjusted the Wilcoxon p -values for multiple comparisons. The overall analysis ranked the three SRNs and the persistence baseline in 50 blocks. Each block is one combination of dataset and seed. The 10 blocks of one dataset share the same data. The overall analysis is therefore optimistic. The Friedman test on the five dataset-level mean RMSEs is reported as well. The best model per dataset is the model with the lowest mean test RMSE. A model is called significantly best only when the tests support the claim.

VII. RESULTS

This section presents the empirical results. The section covers the selected hyper-parameters, the learning curves, the test-set accuracy and the statistical comparison.

A. Selected Hyper-Parameters

Table IV lists the configuration selected for each SRN and dataset.

Three patterns in Table IV deserve their own comment. First, CV selected the largest network of 32 hidden units in nine of the 15 configurations. The larger MRNs therefore contained up to 3265 parameters. The corresponding Elman networks contained at most 1121 parameters. Second, all three Mackey-Glass networks selected the longest window of 24 steps and the largest learning rate of 0.01. The Jordan network selected the shortest window of six steps for the Airline and S&P 500 series. Third, the MRN selected three memory banks with decay rates of 0.25, 0.5 and 0.75 for the Temperature, Mackey-Glass and S&P 500 series. The MRN selected a single bank with a decay rate of 0.5 for the Airline and Sunspots series. The CV standard deviations of the Airline and S&P 500 series are large relative to the means. The folds of both series span periods with very different levels.

B. Learning Curves

Fig. 3 shows the training and early-stopping loss of the final models trained with seed 0.

The training and early-stopping curves of the Airline, Temperature and Mackey-Glass models decreased together. The joint decrease shows that the SRNs learnt real structure. On the Airline and Temperature series, early stopping halted training once the early-stopping loss flattened. The training loss was still falling slightly at the halting point. A flat early-stopping loss with a falling training loss marks the onset of overfitting. The curves reveal three further behaviours:

- *Mackey-Glass*: The two curves decreased together over the first 100 epochs. The curves oscillated because CV selected the largest learning rate of 0.01 for all three Mackey-Glass networks. Early stopping ended the Mackey-Glass runs well before the limit of 1000 epochs. The median best epoch lay between 94 and 170, as Table IV shows. The Mackey-Glass models therefore converged within the available epochs.
- *Sunspots*: The early-stopping loss lay below the training loss. The early-stopping period from 1909 to 1936 contains weak solar cycles. The mean of the early-stopping period was 37.6. The mean of the training part was 45.4. The errors in the calmer early-stopping period were smaller in absolute terms. The median best epoch of the Sunspots MRN was only four.
- *S&P 500*: The early-stopping loss was almost flat and well above the training loss. The early-stopping period from 2001 to 2009 contains two market crashes. The median best epoch of the S&P 500 MRN was only six. Early stopping prevented the SRNs from memorising the noise of the returns.

The Jordan curves for the Airline and Temperature series were the noisiest. The Jordan curves also showed the smallest reduction from the initial loss. The small reduction is the first sign of the limited memory of the Jordan network.

TABLE IV
SELECTED HYPER-PARAMETERS, NUMBER OF PARAMETERS AND CROSS-VALIDATION ERROR PER NETWORK

Dataset	Network	n_h	n_w	Learning rate	Weight decay	Decay rates	Parameters	CV RMSE	Best epoch
Airline	Elman	32	24	0.01	0	—	1121	16.19 ± 3.33	31
	Jordan	32	6	0.003	10^{-5}	—	129	27.67 ± 4.47	70
	MRN	32	24	0.01	0	0.5	1153	16.07 ± 2.42	62
Sunspots	Elman	8	12	0.01	10^{-3}	—	89	14.15 ± 1.81	12
	Jordan	4	24	0.003	0	—	17	14.31 ± 1.64	56
	MRN	32	12	0.001	10^{-3}	0.5	1153	14.08 ± 1.78	4
Temperature	Elman	16	24	0.003	0	—	305	2.45 ± 0.09	16
	Jordan	32	12	0.01	0	—	129	2.58 ± 0.11	22
	MRN	32	24	0.001	10^{-3}	0.25, 0.5, 0.75	3265	2.44 ± 0.07	22
Mackey-Glass	Elman	32	24	0.01	0	—	1121	0.0027 ± 0.0010	136
	Jordan	8	24	0.01	0	—	33	0.0199 ± 0.0014	170
	MRN	32	24	0.01	0	0.25, 0.5, 0.75	3265	0.0022 ± 0.0009	94
S&P 500	Elman	4	24	0.001	10^{-4}	—	29	17.74 ± 17.20	68
	Jordan	32	6	0.001	10^{-3}	—	129	17.88 ± 17.42	38
	MRN	8	6	0.01	0	0.25, 0.5, 0.75	241	17.92 ± 17.54	6

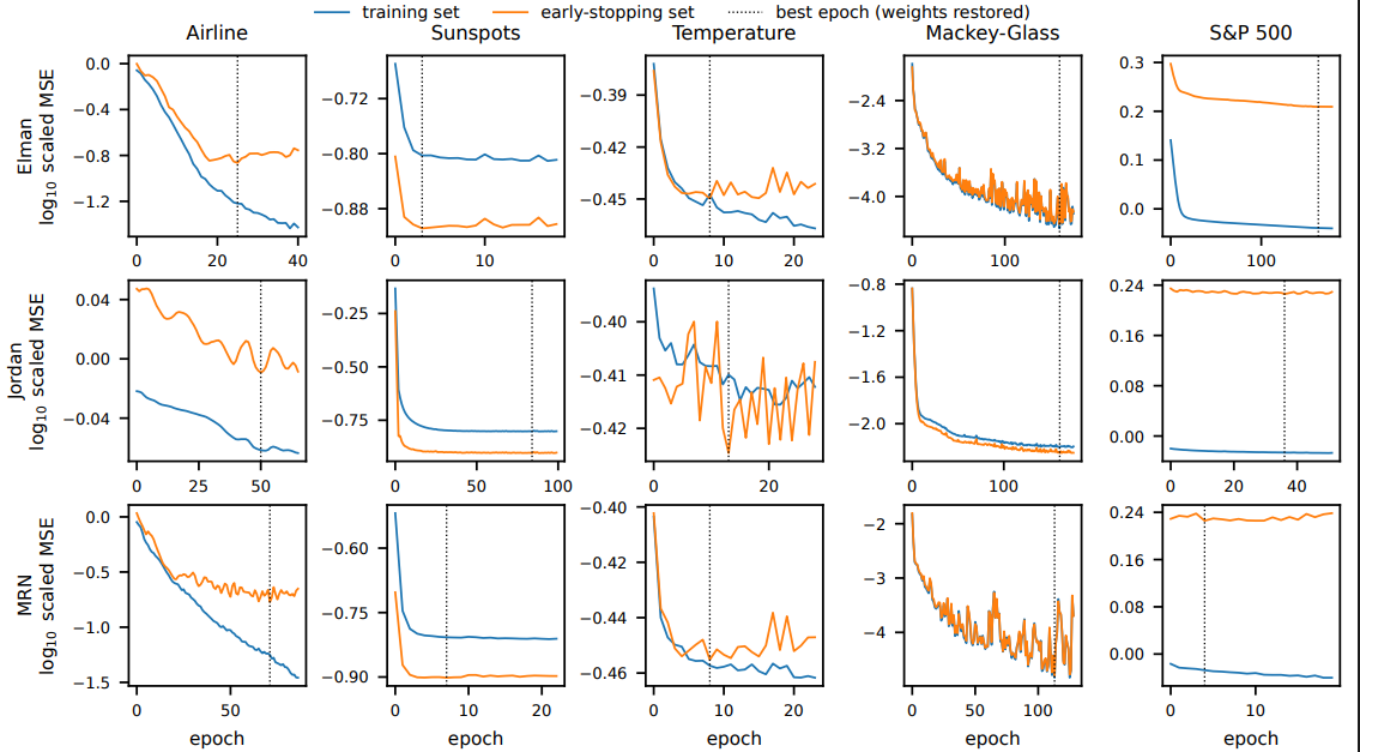


Fig. 3. Training and early-stopping loss curves of the final models trained with seed 0. The dotted line marks the restored epoch.

C. Held-Out Test Set

Table V reports the test-set errors over the 10 seeds.

Each entry in Table V gives the mean and the standard deviation over the 10 seeds. Bold entries mark the lowest mean among the three SRNs. The last column gives the error of the persistence baseline. Fig. 4 shows the spread of the test RMSE across the seeds.

On four of the five datasets, every SRN beat the persistence baseline on every measure. The size of the improvement

depended strongly on the series:

- *Mackey-Glass*: The MRN reduced the RMSE of persistence by 97% from 0.0330 to 0.0011. The Elman network reduced the RMSE of persistence by 96%. The Jordan network reduced the RMSE of persistence by only 46%. The RMSE of the MRN was approximately 20% lower than the RMSE of the Elman network.
- *Airline*: The MRN reduced the persistence RMSE by 46%. The Elman network reduced the persistence RMSE

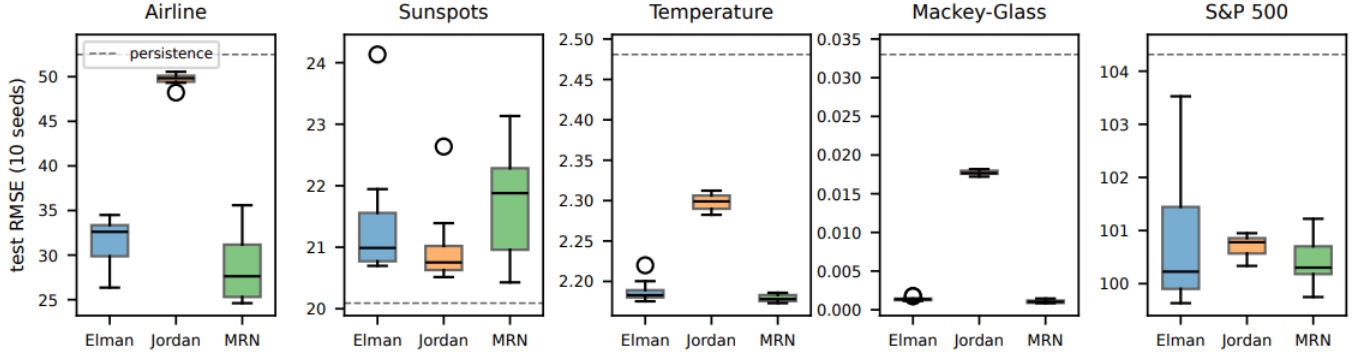


Fig. 4. Test root mean squared error over 10 seeds per network. The dashed line marks the persistence baseline.

TABLE V
TEST-SET ERRORS ON THE ORIGINAL SCALE OVER 10 SEEDS

Dataset	Measure	Elman	Jordan	MRN	Persist.
Airline	RMSE	31.56±2.53	49.73±0.64	28.49±3.45	52.49
	MAE	22.04±1.51	41.01±0.65	21.79±2.47	44.72
	sMAPE (%)	4.71±0.33	9.17±0.15	4.74±0.53	10.09
Sunspots	RMSE	21.38±1.00	20.99±0.60	21.78±0.90	20.09
	MAE	15.17±0.47	14.86±0.33	15.35±0.42	14.84
	sMAPE (%)	27.90±0.31	27.43±0.20	27.97±0.23	28.99
Temperature	RMSE	2.19±0.01	2.30±0.01	2.18±0.00	2.48
	MAE	1.73±0.01	1.82±0.01	1.72±0.00	1.95
	sMAPE (%)	17.31±0.11	18.08±0.08	17.22±0.05	19.99
Mackey-Glass	RMSE	0.0014±0.0002	0.0177±0.0003	0.0011±0.0002	0.0330
	MAE	0.0011±0.0002	0.0148±0.0003	0.0008±0.0002	0.0274
	sMAPE (%)	0.12±0.02	1.78±0.03	0.10±0.02	3.29
S&P 500	RMSE	100.7±1.2	100.7±0.2	100.4±0.5	104.3
	MAE	66.13±0.99	65.90±0.17	66.09±0.76	71.89
	sMAPE (%)	2.39±0.04	2.38±0.01	2.40±0.02	2.61

by 40%. The Jordan network improved on persistence by only 5%. The sMAPE of the Jordan network was 9.17%. The Jordan sMAPE was approximately twice the sMAPE of the other two SRNs.

- *Temperature*: The MRN and the Elman network reduced the persistence RMSE by approximately 12%. The Jordan network reduced the persistence RMSE by 7%. The standard deviation of the series is 4.07 degrees Celsius, as Table I shows. The small improvement indicates that day-to-day weather noise dominates the series.
- *S&P 500*: All three SRNs were approximately 3.5% better than persistence in RMSE. All three SRNs were approximately 8% better than persistence in MAE. The three SRNs were almost indistinguishable from each other.
- *Sunspots*: No SRN beat the persistence RMSE of 20.09. The Jordan network matched persistence in MAE. All three SRNs achieved a lower sMAPE than persistence. The sMAPE of the SRNs ranged from 27.4% to 28.0% against 29.0% for persistence.

Fig. 5 compares the forecasts of the most accurate network with the actual values and with persistence.

For the Airline series, the MRN anticipated the seasonal

TABLE VI
STATISTICAL TESTS PER DATASET. E, J AND M DENOTE THE ELMAN, JORDAN AND MULTI-RECURRENT NETWORKS.

Dataset	χ^2_F	p	Ranks E/J/M	Nemenyi	Wilcoxon p (E-J/E-M/J-M)	Best
Airline	16.80	<0.001	1.8 / 3.0 / 1.2	E-J, J-M	0.006 / 0.049 / 0.006	M
Sunspots	2.60	0.273	2.1 / 1.6 / 2.3	none	0.645 / 0.645 / 0.393	none
Temperature	16.80	<0.001	1.8 / 3.0 / 1.2	E-J, J-M	0.006 / 0.037 / 0.006	M
Mackey-Glass	20.00	<0.001	2.0 / 3.0 / 1.0	J-M	0.006 / 0.006 / 0.006	M
S&P 500	1.40	0.497	1.9 / 2.3 / 1.8	none	1.000 / 1.000 / 0.580	none

turning points. Persistence lagged the turning points by one month. For the Mackey-Glass series, the forecast was indistinguishable from the actual series at the plotted scale. For the Sunspots and S&P 500 series, the network forecasts tracked the series closely. The network forecasts reacted to large moves one step late, as persistence did. Little beyond the last value was therefore predictable in the Sunspots and S&P 500 series.

D. Statistical Comparison

Table VI summarises the statistical tests per dataset.

The column χ^2_F in Table VI gives the Friedman statistic. Table VI reports Nemenyi pairs only when the Friedman test rejected. The Wilcoxon p -values are Holm-corrected. The Friedman test rejected equal performance on the Airline, Temperature and Mackey-Glass series with $p < 0.001$. The Friedman test did not reject on the Sunspots series with $p = 0.273$. The Friedman test did not reject on the S&P 500 series with $p = 0.497$. The Nemenyi CD for three methods and 10 blocks is 1.05. The Nemenyi test found the MRN significantly better than the Jordan network on all three series with a Friedman rejection. The Nemenyi test found the Elman network significantly better than the Jordan network on the Airline and Temperature series. The Nemenyi test did not separate the MRN from the Elman network on any dataset. The Wilcoxon tests separated the MRN from the Elman network on all three series with a Friedman rejection. The Wilcoxon p -values were 0.049 for the Airline series, 0.037 for the Temperature series and 0.006 for the Mackey-Glass series.

Fig. 6 shows the overall comparison over all 50 blocks with persistence included.

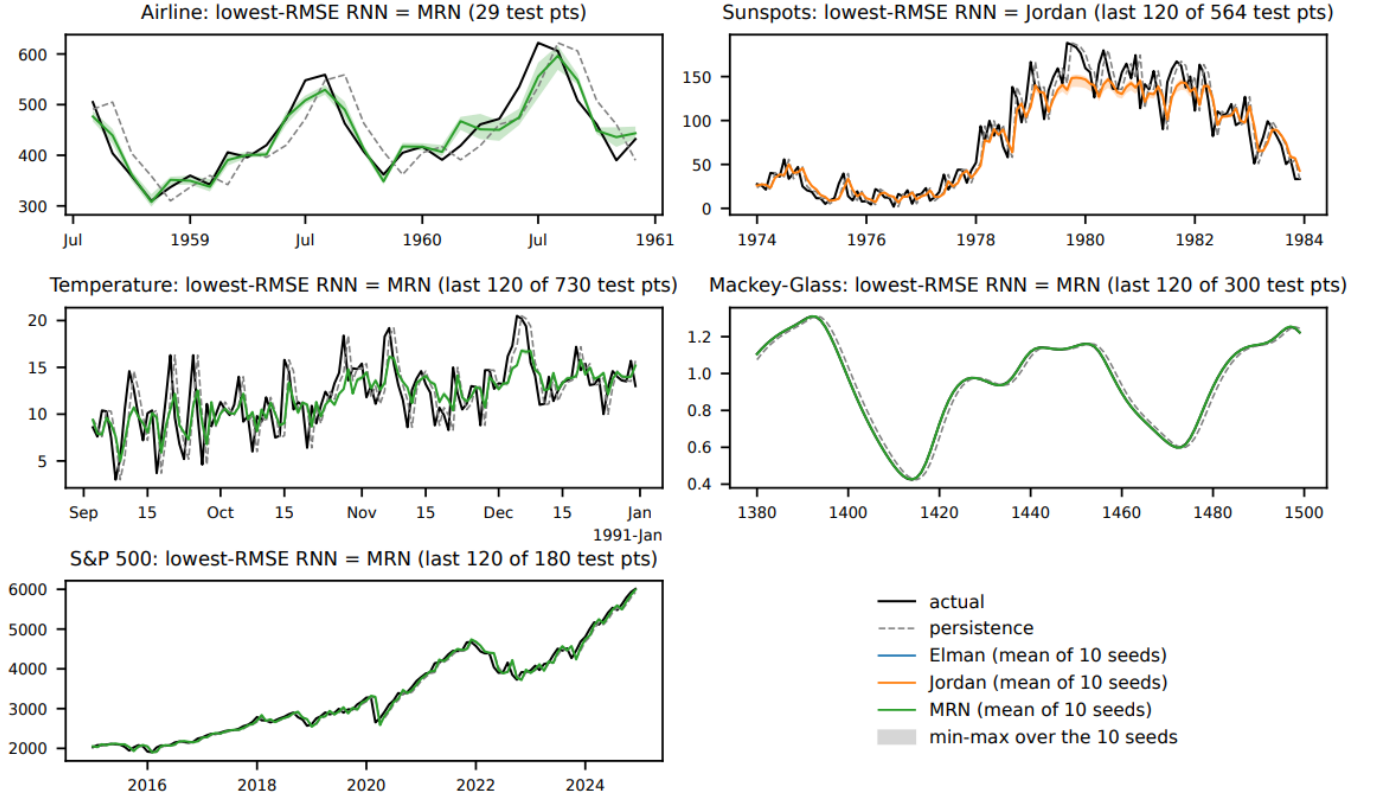


Fig. 5. One-step-ahead test forecasts of the most accurate network per dataset. The shaded band spans the minimum and the maximum over the 10 seeds.

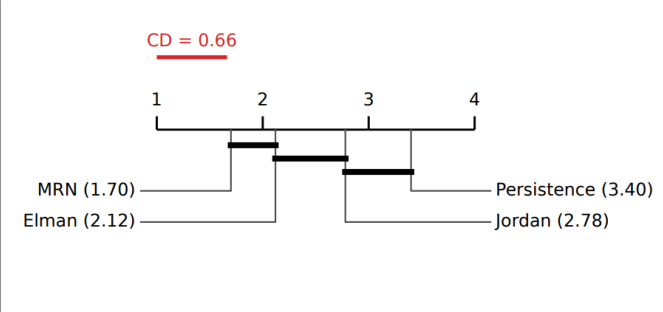


Fig. 6. Critical-difference diagram of the average ranks over all dataset and seed combinations. A thick bar joins methods without a significant difference.

The Friedman test rejected equal ranks with $p = 7.3 \times 10^{-11}$. The MRN had the best average rank of 1.70. The Elman network followed with an average rank of 2.12. The Jordan network and persistence had average ranks of 2.78 and 3.40. The CD for four methods and 50 blocks is 0.663. The MRN was significantly better than the Jordan network and persistence. The Elman network was significantly better than persistence. The rank difference of 0.66 between the Elman and Jordan networks fell just below the CD. The Elman network was therefore not significantly better than the Jordan network. The MRN and the Elman network were not significantly different from each other. The Jordan network

was not significantly better than persistence.

The ranking of the five dataset-level mean RMSEs gave the same order. The MRN, the Elman network, the Jordan network and persistence had average ranks of 1.6, 2.4, 2.6 and 3.4. The Friedman test with only five blocks had little power. The Friedman test did not reject with $\chi^2_F = 4.92$ and $p = 0.178$. The corresponding CD is 2.10.

VIII. DISCUSSION

This section interprets the results. The section discusses the role of the memory structure and the series with little predictable structure. The section then discusses under- and overfitting, the limitations of the study and future work.

A. Memory Structure of the Three Networks

The results followed the amount and type of memory of each SRN. The Jordan network remembers only one number. The number is the previous forecast of the Jordan network. The previous forecast estimates the current input. The Jordan network already receives the current input. The feedback of the Jordan network therefore adds little new information. The Jordan network behaves almost like a feed-forward network on the input window. The limited memory explains the failure of the Jordan network on the Airline and Mackey-Glass series. Both series require a detailed internal state. The next Mackey-Glass value depends on the state of the system 17 time steps earlier.

The Elman context holds a learned summary of the past with n_h dimensions. The MRN adds copies of the hidden and output activations with different decay rates. The decay rates give the MRN memory on multiple time scales at once. The MRN selected three banks for the Mackey-Glass and Temperature series.

The Airline series provides the cleanest comparison of the memory structures. The Airline MRN and the Airline Elman network had 1153 and 1121 parameters. The MRN nevertheless achieved a lower RMSE of 28.49 against 31.56. The Wilcoxon test confirmed the difference with $p = 0.049$. The Mackey-Glass and Temperature comparisons are less clean. The MRNs for both series had 3265 parameters. The corresponding Elman networks had 1121 and 305 parameters. Part of the advantage of the MRN on the two series therefore reflects the larger model.

B. Series with Little Predictable Structure

The three SRNs were statistically indistinguishable on the Sunspots and S&P 500 series. For the S&P 500 series, the persistence baseline was already hard to beat. The SRNs improved on persistence by only approximately 3.5% in RMSE. The architecture of the SRN therefore hardly mattered.

The Sunspots series is more instructive. The test period from 1937 to 1983 contains the strongest solar cycles of the record. The test period has a mean of 77.7 and a standard deviation of 55.0. The development data has a mean of 44.7 and a standard deviation of 37.2. The maximum of the test period exceeds the maximum of the development data. The standardisation parameters were fitted to the calmer development regime. The hyperbolic tangent units are bounded. The SRNs therefore under-predicted the peaks of the test period, as Fig. 5 shows. Persistence adapts to any level automatically. Persistence consequently won on RMSE. Large errors at the peaks dominate the RMSE. The SRNs still won on sMAPE. The sMAPE gives more weight to the quiet periods.

C. Under- and Overfitting

The protocol controlled overfitting effectively. Early stopping ended every run well before the epoch limit, as the best epochs in Table IV show. The learning curves in Fig. 3 showed no late divergence between the training and early-stopping loss. The spread over seeds was small. The standard deviation of the RMSE was at most 1.2% of the mean on the Temperature and S&P 500 series.

The higher epoch limit removed the under-training of the Mackey-Glass series. The preliminary run with a limit of 300 epochs had median best epochs between 296 and 298. The final Mackey-Glass runs stopped early at median best epochs between 94 and 170. The higher limit and the larger search together reduced the Mackey-Glass RMSE of the MRN from 0.0016 to 0.0011. The same changes reduced the Mackey-Glass RMSE of the Elman network from 0.0029 to 0.0014. CV selected a weight decay of 0 or 10^{-5} for 10 of the 15 selected configurations. Early stopping and capacity selection

therefore provided sufficient regularisation for SRNs of the tested size.

D. Limitations

The random search evaluated 24 of the 144 base configurations per SRN and dataset. The selected MRNs contained up to three times as many parameters as the selected Elman networks. The comparison therefore does not fully separate the memory structure from the model size. The overall significance analysis treats the 10 seeds of a dataset as separate blocks. The blocks of one dataset share the same data. The overall result is therefore optimistic. The dataset-level test with five blocks did not reject. The study also evaluated only one-step-ahead forecasts.

E. Future Work

Four extensions follow from the limitations. First, a comparison with an equal number of parameters per SRN would separate the effect of the memory structure from the effect of model size. Second, a square-root transform would stabilise the variance of the Sunspots series. Re-fitting the standardisation at each forecast origin is an alternative for series with a drifting level. Third, multi-step forecasting would use the output feedback of the Jordan network in a closed loop. Multi-step forecasting would therefore test the Jordan network under conditions closer to the original design. Fourth, a Jordan network with a self-connection on the state unit would test the original formulation of Jordan.

IX. CONCLUSION

The goal of this study was to compare the Elman network, the Jordan network and the multi-recurrent network (MRN) for one-step-ahead forecasting. The three simple recurrent neural networks (SRNs) were implemented from the defining equations. The SRNs were compared on five time series. The procedure respected the order of time throughout. The procedure used a chronological held-out test set and five-fold expanding-window cross-validation. A random search over 24 configurations with two tuning seeds selected the hyper-parameters. Training-only scaling, 10 seeds per model and non-parametric significance tests completed the procedure.

The MRN was the most accurate SRN overall. The MRN achieved the lowest mean test root mean squared error (RMSE) on the Airline, Temperature and Mackey-Glass series. The MRN also achieved the best average rank of 1.70. The MRN was significantly better than the Jordan network and the persistence baseline. The margin of the MRN over the Elman network was significant only under the Wilcoxon tests. Part of the margin on the Temperature and Mackey-Glass series reflected the larger size of the MRN. The Jordan network was consistently the weakest SRN. The single fed-back output of the Jordan network carried little information beyond the latest input. The choice of SRN made no significant difference on the Sunspots and Standard and Poor's 500 series. No SRN beat persistence in RMSE on the Sunspots series. A shift in level and variance between the development and test periods

explained the Sunspots result. Coupled together, the results supported the MRN as the recommended SRN for the task. The Elman network was a close second.

Beyond the specific outcome, the study offered concrete illustrations of three theoretical points. The first point was the value of memory on multiple time scales for seasonal and chaotic series. The second point was the limited information carried by output-only feedback. The third point was the difficulty of bounded networks when test data left the range of the training data.

APPENDIX A ACRONYMS

ADF	augmented Dickey-Fuller
CD	critical difference
CV	cross-validation
KPSS	Kwiatkowski-Phillips-Schmidt-Shin
MAE	mean absolute error
MRN	multi-recurrent network
MSE	mean squared error
RMSE	root mean squared error
S&P 500	Standard and Poor's 500
sMAPE	symmetric mean absolute percentage error
SRN	simple recurrent neural network

APPENDIX B SYMBOLS

a_j	decay rate of memory bank j
$\mathbf{b}$	hidden bias vector
$\mathbf{c}_t$	context vector of the Elman network
d	differencing order
$\mathbf{G}$	state-to-hidden weight matrix
$\mathbf{h}_t$	hidden state at time step t
i	fold index
j	memory bank index
k	number of folds in standard cross-validation
$\mathbf{m}_{j,t}$	hidden memory of bank j
n_b	number of memory banks
n_d	number of development targets
n_h	number of hidden units
n_i	number of inputs to a layer
n_m	number of compared methods
n_s	number of blocks
n_w	window length
o_t	observed value at time step t
$\hat{o}_t$	forecast of o_t on the original scale
$\mathbf{Q}$	memory-to-hidden weight matrix
q_α	critical value for the critical difference
$r_{j,t}$	output memory of bank j
s_t	state unit of the Jordan network
T	set of test time steps
t	time step
$\mathbf{U}$	context-to-hidden weight matrix
u_t	transformed series
$\hat{u}_t$	predicted value of u_t
$\mathbf{V}$	hidden-to-output weight matrix
$v(t)$	state of the Mackey-Glass system
$\mathbf{W}$	input-to-hidden weight matrix
x_t	input at time step t
y_t	output at time step t
$\mathbf{z}_t$	concatenated memory vector of the MRN
α	significance level
β	output bias
χ_F^2	Friedman test statistic
δ	Euler integration step size
τ	Mackey-Glass delay

REFERENCES

- [1] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [2] C. Ulbricht, "Multi-recurrent networks for traffic forecasting," in *Proc. 12th Nat. Conf. Artificial Intelligence (AAAI-94)*, 1994, pp. 883–888.
- [3] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.
- [4] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proc. IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.
- [5] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, 1994.
- [6] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.
- [7] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: An analysis and review," *Int. J. Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.

- [8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [9] Soumya, "Cross-validation in time series," Medium. [Online]. Available: <https://medium.com/@soumyachess1496/cross-validation-in-time-series-566ae4981ce4>
- [10] V. Cerqueira, L. Torgo, and I. Mozetič, "Evaluating time series forecasting models: An empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020.
- [11] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*, rev. ed. San Francisco, CA, USA: Holden-Day, 1976.
- [12] J. Brownlee, "Datasets for machine learning." [Online]. Available: <https://github.com/jbrownlee/Datasets>
- [13] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root," *J. Econometrics*, vol. 54, no. 1–3, pp. 159–178, 1992.